Lab 05

COMP1322 Programming II

Student Name: Qi Liu
Student ID: 36557854
Student Email: ql1e24@soton.ac.uk

Date: March 17, 2026

1.Preparation

1. Read through the - hello-view.fxml (optional) and record your understanding in the logbook

```
<?xml version="1.0" encoding="UTF-8"?> <!-- declare this is an XML file and
uses UTF-8 encoding -->

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.control.Button?>
<!-- import JavaFX classes used in this UI -->

<VBox alignment="CENTER" spacing="20.0"
      xmlns:fx="http://javafx.com/fxml"
      fx:controller="com.example.javafxdemo.HelloController">
   <!-- VBox is a layout container that arranges items vertically -->
   <!-- alignment="CENTER" means all items will be centered -->
   <!-- spacing="20.0" means there is 20 pixels space between components -->
   <!-- fx:controller connects this UI to the HelloController Java class -->

   <padding>
       <Insets bottom="20.0" left="20.0" right="20.0" top="20.0"/>
   </padding>
   <!-- adds 20 pixels of space around the edges -->

   <Label fx:id="welcomeText"/>
   <!-- display text -->
   <!-- fx:id allows the controller to access this label in Java code -->

   <Button text="Hello!" onAction="#onHelloButtonClick"/>
   <!-- onAction calls the method onHelloButtonClick() in the controller when
clicked -->

</VBox>
       <!-- End of the VBox layout -->
```

2. Drag and drop and explore the available options – and check what are the reflections in the code file – hello-view.fxml. Record your observation in the logbook.

1. Adding Components

New XML tags are automatically inserted in the `hello-view.fxml` file. Each component created in the user interface using Scene Builder is reflected as a corresponding XML tag in the FXML code.

2. Changing Properties
Updating component properties in the XML structure.

3. Layout Changes
Updating layout container properties in the XML structure.

4. Adding fx:id for Controller Access
The component is accessible in the controller class using the `@FXML` annotation.

5. Event Handling
The event will be linked to the component in the controller class using the `onAction` attribute in the XML structure.

2. Programming Task

1. What are the benefits of using nested layouts in JavaFX?
Nested layouts are used to arrange a number of layout panes to make the user interface more manageable. By placing a layout inside another layout, making it easier to manage different parts of a window.

2. How does GridPane handle element placement and resizing?
It organizes components in a grid of rows and columns. Each component is placed at a specific row index and column index. When the window is resized, it can resize components automatically.

3. Which layout pane would be best for a dynamic UI with changing elements?
VBox or HBox can easily add or remove components. It automatically arranges components.

✎ Logbook Entry After completing this exercise, write a short logbook entry including:
1. Understanding of Layout Panes: Summarize what you learned about different JavaFX layout panes.
JavaFX provides layout panes such as VBox, HBox, GridPane, and BorderPane to organize UI components. Each layout arranges elements differently, which gives me a strong feeling of HTML and CSS, even though I know it is not the same type of languages, but they all provide me chances to edit my own GUI program.

2. Challenges Faced: Describe any difficulties encountered and how you solved them.

At first, it was difficult to understand how layout panes control the position of elements correctly. But by observing how the FXML code changed when I modified the layout with the help of Scene Builder I learned a lot.

3. Improvements: What would you do differently if you had to build a more complex UI?

For a more complex UI, I would draw the layout structure first on my own drawing board.

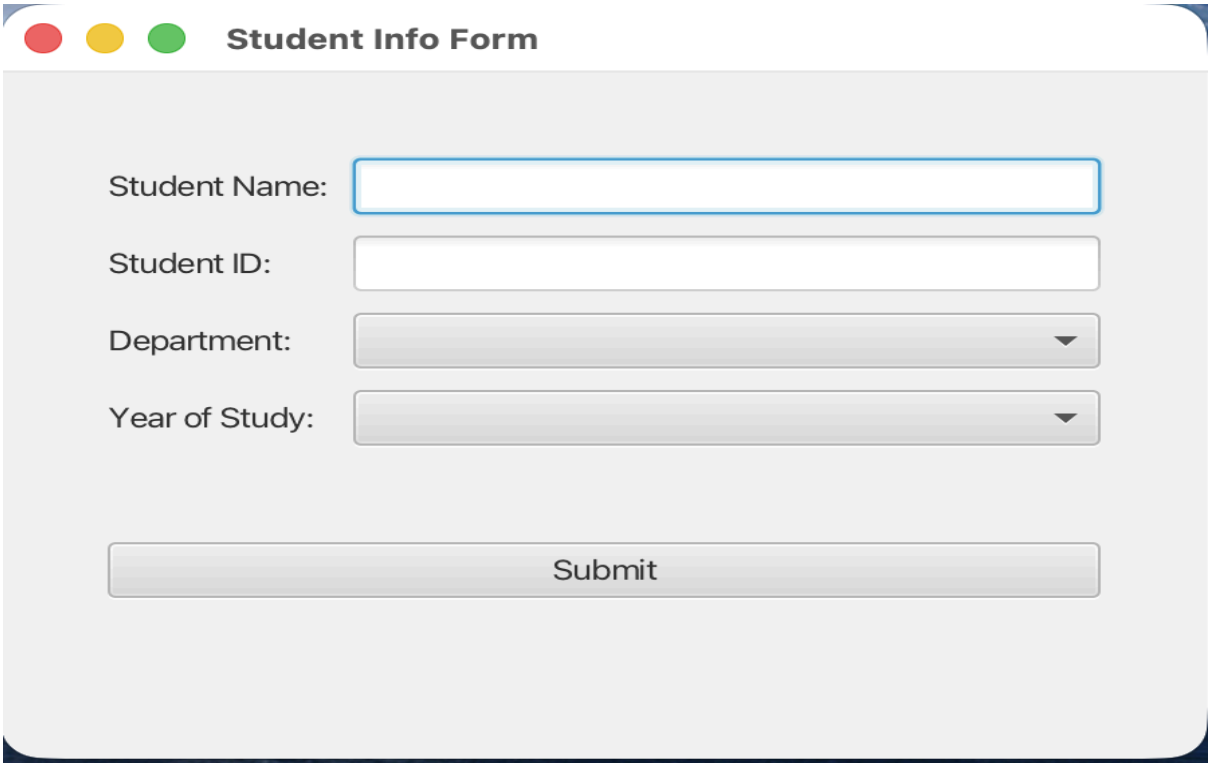
Code:

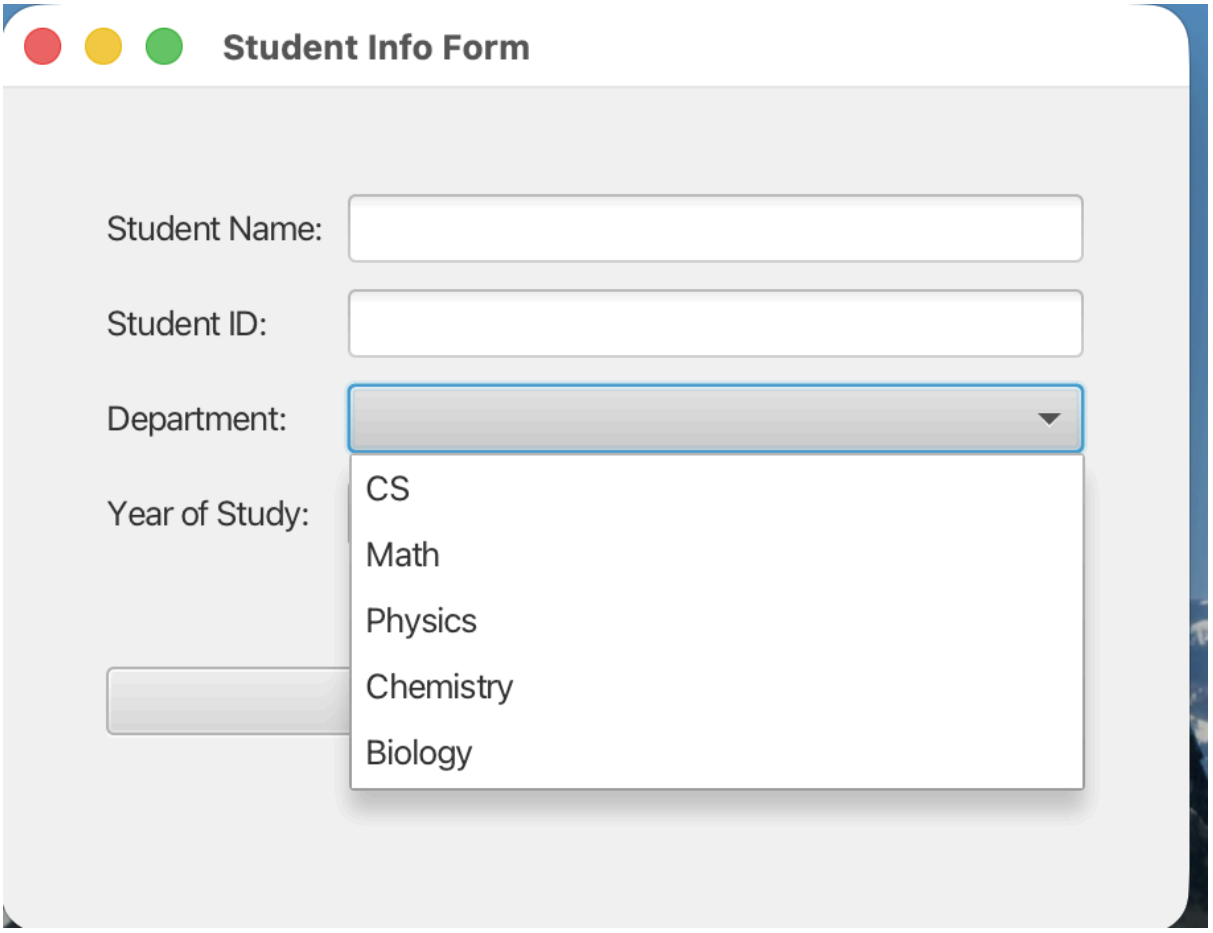
```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.ChoiceBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.Priority;
import javafx.scene.layout.StackPane;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class StudentInfoForm extends Application {

   @Override
   public void start(Stage primaryStage) {
       // basic labels for each field
       Label nameLabel = new Label("Student Name:");
       Label idLabel = new Label("Student ID:");
       Label deptLabel = new Label("Department:");
       Label yearLabel = new Label("Year of Study:");

       // text boxes for user input
       TextField nameField = new TextField();
       TextField idField = new TextField();
       nameField.setMaxWidth(Double.MAX_VALUE);
       idField.setMaxWidth(Double.MAX_VALUE);

       // several dropdown list for department
       ComboBox<String> deptComboBox = new ComboBox<>();
       deptComboBox.getItems().addAll("CS", "Math", "Physics", "Chemistry",
"Biology");
       deptComboBox.setMaxWidth(Double.MAX_VALUE);
```

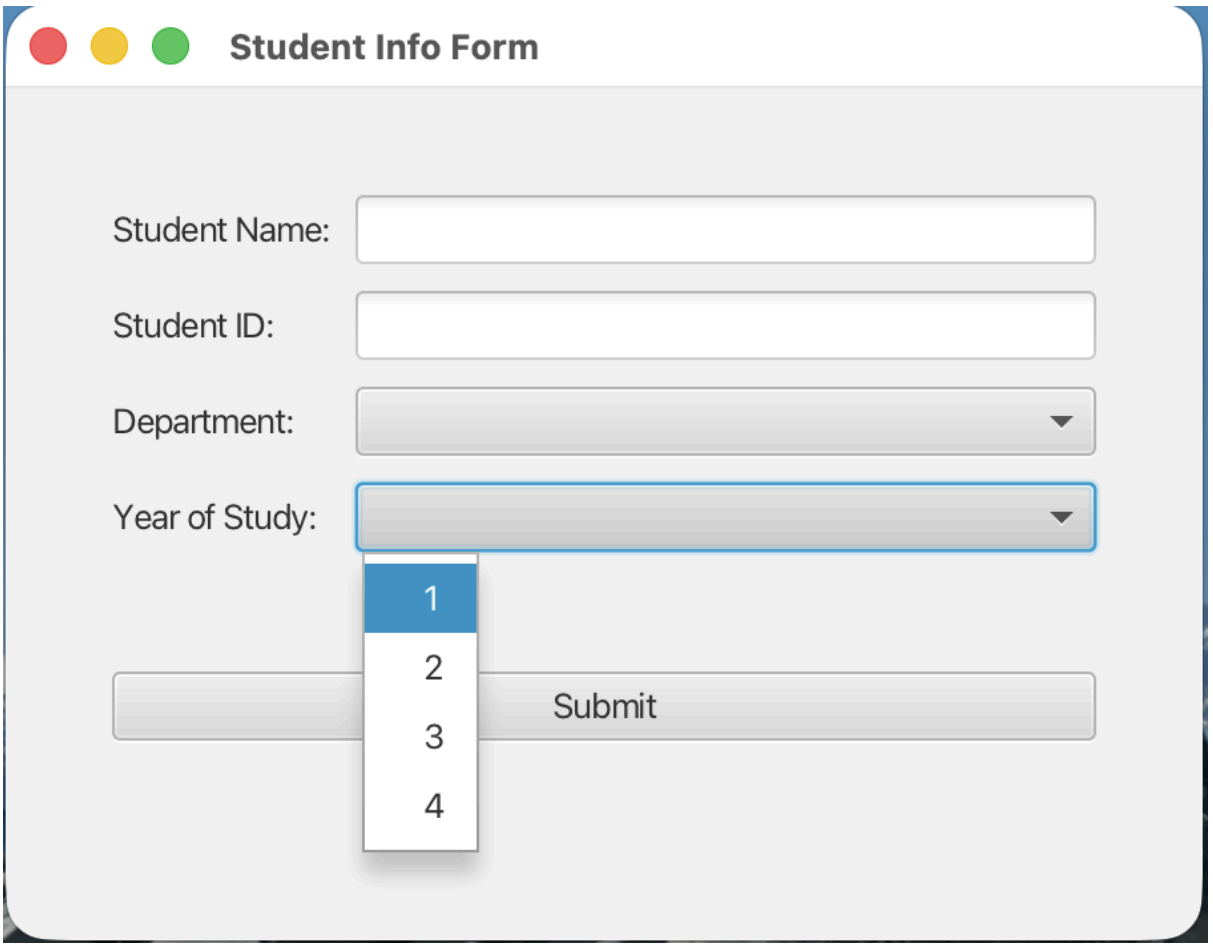
```
        // choice box for study year
        ChoiceBox<String> yearChoiceBox = new ChoiceBox<>();
        yearChoiceBox.getItems().addAll("1", "2", "3","4");
        yearChoiceBox.setMaxWidth(Double.MAX_VALUE);

        GridPane grid = new GridPane(); // arrange the form in rows and
columns
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(20));

        // put labels in the first column and inputs in the second column
        grid.add(nameLabel, 0, 0);
        grid.add(nameField, 1, 0);
        grid.add(idLabel, 0, 1);
        grid.add(idField, 1, 1);
        grid.add(deptLabel, 0, 2);
        grid.add(deptComboBox, 1, 2);
        grid.add(yearLabel, 0, 3);
        grid.add(yearChoiceBox, 1, 3);

        // make the second column grow when the window becomes larger
        GridPane.setHgrow(nameField, Priority.ALWAYS);
        GridPane.setHgrow(idField, Priority.ALWAYS);
        GridPane.setHgrow(deptComboBox, Priority.ALWAYS);
        GridPane.setHgrow(yearChoiceBox, Priority.ALWAYS);

        // button to submit the form
        Button submitButton = new Button("Submit");
        submitButton.setMaxWidth(Double.MAX_VALUE);

        submitButton.setOnAction(event -> System.out.println("Form
submitted!")); // if button is clicked

        StackPane buttonPane = new StackPane();
        buttonPane.getChildren().add(submitButton);
        buttonPane.setPadding(new Insets(10, 20, 10, 20)); // put the button
in the center

        VBox vbox = new VBox();
        vbox.setSpacing(15);
        vbox.setPadding(new Insets(20)); // puts the form and button from top
to bottom
        vbox.getChildren().add(grid);
        vbox.getChildren().add(buttonPane);

        BorderPane root = new BorderPane();
        root.setCenter(vbox); // main layout of the window

        Scene scene = new Scene(root, 450, 320);
        primaryStage.setTitle("Student Info Form");
        primaryStage.setScene(scene);
```

```
        primaryStage.show(); // create the scene and show the stage
    }

    public static void main(String[] args) {
        // Start the JavaFX program
        launch(args);
    }
}
```

Output:


Student Info Form
Student Name:
Student ID:
Department:
Year of Study:
Submit


Student Info Form
Student Name:
Student ID:
Department:
Year of Study:
CS
Math
Physics
Chemistry
Biology


Student Info Form
Student Name:
Student ID:
Department:
Year of Study:
1
2
3
4
Submit